

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:50

Annexure A

Debugging or Optimization task

Code of Conduct:

I Srinivas S certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Q1. Deadlock Debugging (Code Trace)

You are given the following pseudo-code for two processes sharing resources R1 and R2:

Process P1:

```
lock(R1)
wait(100 ms)
lock(R2)
critical section
unlock(R2)
unlock(R1)
```

Process P2:

```
lock(R2)
wait(100 ms)
lock(R1)
critical section
unlock(R1)
unlock(R2)
```

Identify the exact condition that causes deadlock.

Suggest an optimisation to eliminate the deadlock without changing process logic.

(10 Marks)

A.

1. Exact condition causing deadlock

Deadlock occurs because:

- **P1 locks R1** and waits for **R2**.
- **P2 locks R2** and waits for **R1**.

This creates a **circular wait**, where each process waits for a resource held by the other.

2. Optimization to eliminate deadlock

Acquire resources in the **same order** for all processes.

Example:

P1: lock(R1) → lock(R2)

P2: lock(R1) → lock(R2)

Using a fixed resource ordering removes the circular wait condition and prevents deadlock.

Q2. CPU Utilisation Problem

A system shows low CPU utilisation (30%) even though processes are ready. You analyse the scheduler log and find frequent transitions:

ready → waiting → ready → waiting → ready ...

Debug the cause using process management concepts and propose a system-level optimisation to improve utilisation.

(10 Marks)

A.

Cause

The scheduler log shows:

Ready → Waiting → Ready → Waiting

This indicates that processes are frequently waiting for **I/O operations** instead of using the CPU.

As a result, the CPU remains idle even though processes exist.

System-level optimization

- Use **asynchronous I/O** or improve I/O scheduling.
- Increase multiprogramming so another ready process can execute while one process waits for I/O.

Result: CPU utilization increases because the CPU spends less time idle.

Q3. Memory Bottleneck Debugging

A multi-threaded program frequently encounters page faults, slowing performance. The memory map shows:

- Code: 5 MB
- Heap: 150 MB
- Stack: 1 MB per thread
- 25 threads running
- Physical RAM available: 2 GB

Identify the memory issue causing performance degradation and propose two optimisations.

(10 Marks)

A.

Memory issue

The application has:

- Code = **5 MB**
- Heap = **150 MB**
- Stack = **1 MB × 25 threads = 25 MB**
- RAM = **2 GB**

The frequent **page faults** indicate that memory accesses are causing excessive paging, slowing performance.

Two optimizations

1. Reduce the number of threads or decrease the stack size for each thread.
2. Optimize memory usage by reducing heap allocation and reusing memory where possible.

Result: Fewer page faults and improved performance.

Q4. Starvation Debugging

You observe that process P5 never gets CPU time in a multilevel queue scheduler:

- Q1 (Highest Priority): Interactive
- Q2: System
- Q3: Batch
- Time Quantum: Q1 = 5ms, Q2 = 10ms, Q3 = FCFS

P5 is in Q3 but keeps getting postponed.

Debug the root cause of starvation and suggest an optimisation to ensure fairness.

(10 Marks)

A.

Root cause

Process **P5** is in **Q3 (Batch Queue)**.

Higher-priority queues (**Q1 and Q2**) continuously receive processes, so Q3 rarely gets CPU time.

Hence, **P5 experiences starvation.**

Optimization

Use **Aging**.

Increase the priority of a process the longer it waits.

Benefits:

- Prevents starvation.
- Ensures fair CPU allocation.
- Every process eventually gets CPU time.

Q5. Context-Switch Overhead Debugging

A server performs 20,000 context switches/sec, causing high overhead.

Log details show:

- Average burst time per thread: 2ms
- I/O wait frequency: very high
- Preemptive scheduler with quantum = 1ms

Explain why context switching is so high and propose two optimisations to reduce overhead.

(10 Marks)

A.

Why is context switching high?

- Time quantum = **1 ms**, while average burst time = **2 ms**.
- The scheduler interrupts processes before they finish.
- High I/O frequency also causes frequent switching.

Hence, about **20,000 context switches per second** occur, increasing CPU overhead.

Two optimizations

1. Increase the time quantum (for example, from **1 ms to 4–5 ms**) to reduce unnecessary preemption.
2. Reduce context switches by improving I/O scheduling or grouping I/O operations.

Result:

- Lower context-switch overhead.
- Better CPU utilization.
- Improved system throughput.

These answers directly address the debugging and optimization tasks described in the uploaded assessment.

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated

Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation
-----------------	-----	---	--	--	---